GIT • GITHUB

Verziókövetés alapjai

A legfontosabb fogalmak és parancsok kezdőknek

Windows PowerShell / bármely terminál • Git 2.x • 2026. november

1. Miért fontos a verziókövetés?

A verziókövetés egy mappa **memóriája**: minden mentett állapotot megjegyez, és megmutatja, mi, mikor, ki által és miért változott. A legelterjedtebb eszköze a **Git**; a **GitHub** (vagy GitLab, Bitbucket) a Git-tárolók online otthona. Bármilyen szöveges fájlon működik: kódon, szkripten, CSV-n, Markdown-dokumentumon. A *jelentés_v3_végleges_JAVÍTOTT.docx* típusú fájlnevek helyett egyetlen fájl van, a változatokat a Git tartja nyilván.

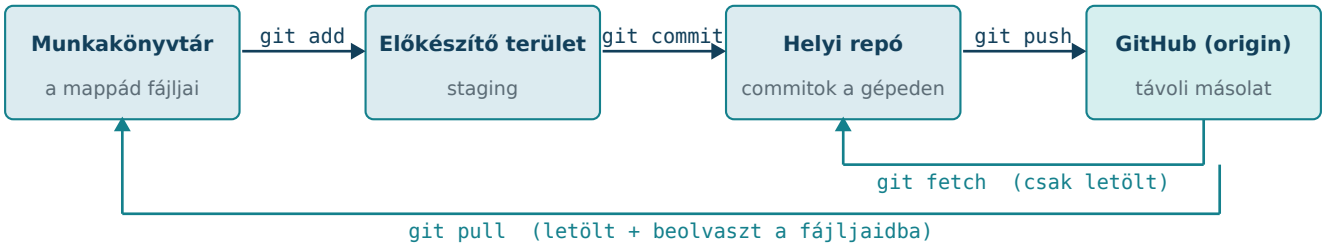
- **Visszaút bármikor.** Minden mentett állapot (commit) visszaállítható; egy elrontott módosítás nem végleges.
- **Látható változások.** Soronként megmutatja, mi módosult (diff) – még a mentés előtt ellenőrizhetsz.
- **Biztonságos kísérletezés.** Egy új ötletet külön ágon (branch) próbálsz ki; ha nem válik be, a fő változat érintetlen marad.
- **Együttműködés.** Többen dolgozhatnak ugyanazon a projekten; a Git összefésüli a változásokat és jelzi az ütközéseket.
- **Átláthatóság és mentés.** Minden lépés visszakövethető (auditálhatóság), a távoli másolat pedig géphiba esetén is megmarad.
- **Biztonsági háló az AI-eszközökhöz.** Ha egy AI-kódoló eszköz módosít fájlokat, előtte commitolva mindig van visszaút, utána a diffben ellenőrizheted a munkáját.

2. Alapfogalmak

Fogalom	Jelentése
Repó (repository)	Mappa verziótörténettel (rejtett **.git** almappa). Lehet helyi (a gépeden) és távoli (pl. GitHubon).
Commit	Elnevezett pillanatkép a fájlok állapotáról, rövid üzenettel (mit és miért).
Előkészítő terület (staging)	Ide gyűjtöd a **git add**-dal a következő commitba szánt változásokat.
Ág (branch)	A történet egy önálló szála. Az alapértelmezett ág neve általában **main** (régebben master).
Checkout / switch	Átváltás egy másik ágra (vagy korábbi állapotra): a mappa fájljai az adott ág állapotát mutatják.
Remote	Egy távoli repó elnevezett címe. A klónozott forrás neve alapból **origin**; forknál az eredeti repó szokásos neve **upstream**.
Clone	Egy távoli repó teljes letöltése a gépedre, a teljes történettel együtt.
Fork	Más felhasználó repójának saját másolata a *GitHub-fiókodban*. Nem Git-parancs, hanem GitHub-funkció; a forkot utána klónozod. Változtatásaidat **pull requesttel** ajánlhatod fel az eredeti repónak.
Merge	Két ág változásainak összefésülése egy ágba.

Hogyan mozognak a változások?

git stash: nem commitolt módosítások ideiglenes félretétele – git stash pop: visszahozás



3. A legfontosabb parancsok

Parancs	Mire jó?	Példa / megjegyzés
Kezdés		
`git config --global`	Név és e-mail beállítása (egyszer, gépenként).	git config --global user.name "Kiss Anna"
`git init`	Új, üres repó létrehozása az aktuális mappában.	Létrehozza a rejtett .git mappát.
`git clone <URL>`	Távoli repó letöltése a gépre.	git clone https://github.com/felh/projekt.git
Napi ciklus		
`git status`	Mi változott, mi van előkészítve, melyik ágon vagy?	A leggyakoribb parancs – futtasd sokszor.
`git diff`	Még nem előkészített változások soronként.	git diff --staged: az előkészítettek. Kilépés: q
`git add <fájl>`	Változás előkészítése a commitra.	git add jelentes.md • git add . (minden)
`git commit -m "..."`	Pillanatkép készítése üzenettel.	git commit -m "Bevezető átírása"
`git log --oneline`	A commitok listája röviden.	git log --oneline --graph --all: ágakkal. Kilépés: q
Ágak		
`git branch`	Ágak listázása; a * jelöli az aktuálisat.	git branch uj-ag (létrehoz) • git branch -d uj-ag (töröl)
`git checkout <ág>`	Átváltás egy másik ágra.	git checkout -b uj-ag: létrehoz és átvált egyszerre
`git switch <ág>`	Az átváltás újabb, egyértelműbb parancsa.	git switch -c uj-ag: létrehoz és átvált
`git merge <ág>`	A megadott ág beolvasztása az aktuális ágba.	git switch main → git merge uj-ag
Távoli repók		
`git remote -v`	A beállított távoli repók és címeik.	Klónozás után: origin
`git remote add <név> <URL>`	Új távoli repó hozzáadása.	git remote add upstream https://github.com/eredeti/projekt.git
`git fetch`	Letölti a távoli újdonságokat, de **nem** módosítja a fájljaidat.	git fetch upstream • utána: git status
`git pull`	Letölti **és** beolvasztja a távoli változásokat (fetch + merge).	Munka elején érdemes futtatni.
`git push`	Helyi commitok feltöltése.	Új ág első feltöltése: git push -u origin uj-ag
Félretétel és visszavonás		
`git stash`	Nem commitolt módosítások ideiglenes félretétele.	git stash list • git stash pop (visszahozás)
`git restore <fájl>`	Egy fájl nem commitolt módosításainak **eldobása**.	Visszavonhatatlan! --staged: csak kiveszi az előkészítésből
`git revert <commit>`	Egy korábbi commit hatását egy új commit visszafordítja.	A történetet nem írja át – megosztott ágon is biztonságos.
Fork (GitHub)		
`Fork gomb`	Saját másolat más repójáról a GitHub-fiókodban.	A repó GitHub-oldalán jobb felül: Fork
`gh repo fork <repó> --clone`	Ugyanez parancssorból a GitHub CLI-vel.	Forkol, klónoz és beállítja az upstream remote-ot.

4. Két tipikus munkamenet

a) Új ötlet külön ágon

```
git switch main ; git pull           # friss fő ág
git switch -c uj-otlet              # új ág + átváltás (régi forma: git checkout -b uj-otlet)
# ... fájlok szerkesztése ...
git status ; git diff               # mi változott? átnézés
git add . ; git commit -m "Új ötlet első változata"
git push -u origin uj-otlet         # az ág feltöltése
git switch main ; git merge uj-otlet  # ha bevált: beolvasztás a fő ágba
git push ; git branch -d uj-otlet     # feltöltés, a kész ág törlése
```

b) Közreműködés más repójában (fork)

```
# 1. GitHubon: az eredeti repó oldalán Fork → saját másolat a fiókodban
git clone https://github.com/SAJAT-FELHASZNALO/projekt.git
cd projekt
git remote add upstream https://github.com/EREDETI-TULAJ/projekt.git
git remote -v                       # origin = a te forkod, upstream = az eredeti
git switch -c javitas ; ... ; git commit -m "Elírás javítása"
git push -u origin javitas          # 2. GitHubon: Compare & pull request
git fetch upstream ; git switch main ; git merge upstream/main   # később: forkod frissítése
```

5. Gyakori helyzetek

Helyzet	Teendő
A pull/checkout hibát ad: „Your local changes ... would be overwritten”	`git stash` → `git pull` (vagy ágváltás) → `git stash pop`
A push elutasítva: „rejected ... fetch first”	A távoli ágon újabb commit van: `git pull`, majd újra `git push`.
Rossz fájlt adtam hozzá (még nincs commit)	`git restore --staged <fájl>`
Elrontottam egy fájlt (még nincs commit)	`git restore <fájl>`
Rossz ágon dolgoztam (még nincs commit)	`git stash` → `git switch <jó-ág>` → `git stash pop`
Ütközés (CONFLICT) merge vagy pull után	A fájlban <<<<<<< és >>>>>>> jelek közt mindkét változat látszik. Válaszd ki a jót (pl. VS Code Merge Editor), majd git add + git commit.
„Not a git repository”	Rossz mappában vagy: `cd` a repó mappájába; ellenőrzés: `git rev-parse --show-toplevel`
„detached HEAD” üzenet	Egy korábbi commitot néztél meg, nem ágat. Vissza: `git switch main`

6. Aranyszabályok

- **Commit előtt mindig status és diff.** Amit nem néztél át, az ne kerüljön a történetbe.
- **Kicsi, gyakori commitok, beszédes üzenettel.** „Javítás” helyett: „Táblázat fejlécének javítása”.
- **Kísérletezz ágon,** a fő ág (main) maradjon mindig működő állapotban.
- **Munka elején pull, végén push** – így ritkák az ütközések.
- **Érzékeny adat és jelszó soha ne kerüljön a repóba.** Amit egyszer feltöltöttél, a történetben megmarad. A kizárandó fájlokat a **.gitignore** fájlban sorold fel.
- **Bináris fájlok** (xlsx, docx, pdf) tartalmát a diff nem mutatja; ahol lehet, a forrásukat (szöveg, CSV, szkript) verziózd.
- **Történetet átíró parancsok** (`git reset --hard`, `git push --force`, `git clean`) csak akkor, ha pontosan érted a hatásukat.